

UNIVERSITY OF BARISHAL

LAB REPORT

3D Shooting Game with Dynamic Lighting using OpenGL/GLUT

Course Title	Computer Graphics
Course Code	CSE-4110
Session	2019-20, 2020-21
Session / Semester	7th
Project Title	3D Shooting Game with Dynamic Lighting (OpenGL/GLUT)
Supervisor / Coordinator	Dr. Md. Manjur Ahmed
Date of submission	22-06-2026

Submitted By

No.	Name	Student ID
1	Abdullah Al Ovl	20CSE044
2	Jannatul Ferdous Mim	21CSE045

1. Introduction

This report documents the design, implementation, and evaluation of a 3D Shooting Game developed for the Computer Graphics course project. The application was built in C++ using the OpenGL graphics library together with GLUT (the OpenGL Utility Toolkit) for window management, keyboard input, and the rendering/animation loop.

The scene reconstructs a realistic 3D shooting range containing a room, a moving Olympic-style target, a first-person gun model, a bullet system, and a dynamic hexagonal prism light source. Beyond static object placement, the program implements continuous animation (a moving target and bullet trajectory), an interactive lighting system with a glowing effect and volumetric light beam, and keyboard-driven camera and object controls, satisfying the rotation/translation/scaling and lighting requirements of the course rubric.

2. Objectives

- To apply fundamental OpenGL drawing functions to construct a complete, interactive 3D game scene from primitive shapes.
- To implement keyboard-based interaction for aiming, shooting, camera zooming, and light source manipulation.
- To apply advanced lighting and color blending techniques (additive blending) to achieve a realistic glowing effect and volumetric light beam.
- To implement smooth, time-based animation using a GLUT timer callback for target movement and bullet physics.
- To demonstrate teamwork by dividing modeling, animation, and control responsibilities among group members.

3. Tools and Development Environment

Component	Detail
Programming Language	C++
Graphics Library	OpenGL (legacy fixed-function pipeline)
Windowing / Input Toolkit	GLUT (GL/glut.h, with GLUT/glut.h fallback for macOS)

Rendering Mode	Double-buffered RGB with depth testing (GLUT_DOUBLE GLUT_RGB GLUT_DEPTH)
Window Size	1000 x 700 pixels
IDE	Code::Blocks (with freeglut)

4. Overall Program Structure

The program follows a clean, modular structure in which every object in the game is encapsulated in its own drawing function. This separation keeps the codebase readable, makes individual objects easy to debug or extend, and mirrors good software-engineering practice.

Reusable helper functions reduce code duplication across the project:

- `setLightColor(r, g, b, alpha)` – Adjusts the color intensity of objects based on whether the main light is toggled ON or OFF, simulating ambient room lighting.
- `drawText(text, x, y)` – Renders 2D bitmap text on the screen for the UI (Score, Controls, Light Status).

5. OpenGL Transformation Functions (Rotation, Translation, Scaling)

All three core geometric transformations are used extensively, combined through the OpenGL matrix stack (`glPushMatrix()` / `glPopMatrix()`) so that each object can be positioned independently without affecting others.

5.1 Translation (`glTranslatef`)

Every object in the scene is positioned in 3D space using `glTranslatef(x, y, z)`. For example, the target is placed at `(targetX, targetY, -4)`, the gun is positioned relative to the camera, and the bullet translates dynamically along its calculated trajectory vector.

5.2 Rotation (`glRotatef`)

Rotation is applied to manipulate objects and create dynamic effects: Y-axis `(0,1,0)` is used to rotate the hexagonal prism light source based on user input (L/R keys). The volumetric light beam also rotates synchronously with the prism.

5.3 Scaling (glScalef)

glScalef(x, y, z) is used to stretch and resize primitive shapes into the correct proportions. For instance, the gun body is scaled from a solid cube, and the hexagonal prism is scaled down to fit appropriately beside the gun.

6. Texture and Color Mapping

Rather than image-based texture mapping, the project uses procedural color mapping, alpha blending, and material properties to give each object an appropriate, realistic appearance.

- Olympic Target: Constructed using concentric circles (GL_TRIANGLE_FAN) with distinct colors to simulate a professional shooting target.
- Additive Blending (Glow Effect): The drawHexagonalPrism() function uses glBlendFunc(GL_SRC_ALPHA, GL_ONE) to create a tight, pulsating glowing body effect.
- Volumetric Light Beam: A semi-transparent cone is drawn using GL_TRIANGLE_FAN with fading alpha values, simulating a beam of light cutting through the room.
- Dynamic Ambient Lighting: The setLightColor() function dynamically adjusts the RGB values of all room objects based on the light status (ON/OFF).

7. Objects Implemented (2D/3D Construction)

Object / Function	Description
drawRoom()	Floor, back wall, left wall, right wall, and ceiling forming the shooting range shell using GL_QUADS.
drawTarget()	An Olympic-style bullseye target made of 5 concentric colored rings using GL_TRIANGLE_FAN.
drawGun()	A first-person gun model constructed from a scaled glutSolidCube (body) and a gluCylinder (barrel).

drawBullet()	A yellow glutSolidSphere that travels from the gun to the target when shooting.
drawHexagonalPrism()	A 3D hexagonal prism acting as the light source, featuring different colors on each face and a dynamic additive blending glow effect.
drawLightBeam()	A semi-transparent volumetric light beam originating from the prism.
Aiming Point	A simple green GL_POINTS marker used for aiming, replacing traditional crosshairs for a cleaner UI.
UI Text	2D text overlay displaying the current score, controls, and light status using glutBitmapCharacter.

8. Animation

All animation in the scene is driven by a single master timer callback, `update()`, registered with `glutTimerFunc()` to fire every 16 milliseconds (approx. 60 FPS).

- **Moving Target:** The target continuously moves left and right along the X-axis. When it hits the room boundaries, its direction reverses.
- **Bullet Physics:** When the spacebar is pressed, the bullet's directional vector is calculated, and it translates along this vector in the update function.
- **Hit Detection:** The `checkHit()` function calculates the Euclidean distance between the bullet and the target to register hits and increment the score.

9. Camera Movement and Controls

9.1 Camera Control

The camera is defined by an eye position and a look-at reference point, passed to `gluLookAt()` every frame inside `display()`.

Key	Action	Effect
W	Zoom In	Decreases the zoom variable, moving the camera closer.
S	Zoom Out	Increases the zoom variable, moving the camera further away.
C	Change View	Toggles between First-Person View and a Top-Down View.
Arrow Keys	Aiming	Moves the green aiming dot (aimX, aimY) up, down, left, and right.

9.2 Object / Environment Toggles

Key	Action	Effect
Space	Shoot	Fires the bullet towards the aiming point and plays a system beep sound.
O	Toggle Light	Switches lightOn boolean, turning the prism glow, light beam, and room brightness ON or OFF.
L / R	Rotate Light	Increases/decreases lightAngle, rotating the hexagonal prism and its light beam left or right.

Esc	Reset Score	Resets the player's score to 0.
------------	-------------	---------------------------------

10. Teamwork and Responsibilities

The project was divided into two main parts to ensure efficient development and collaboration:

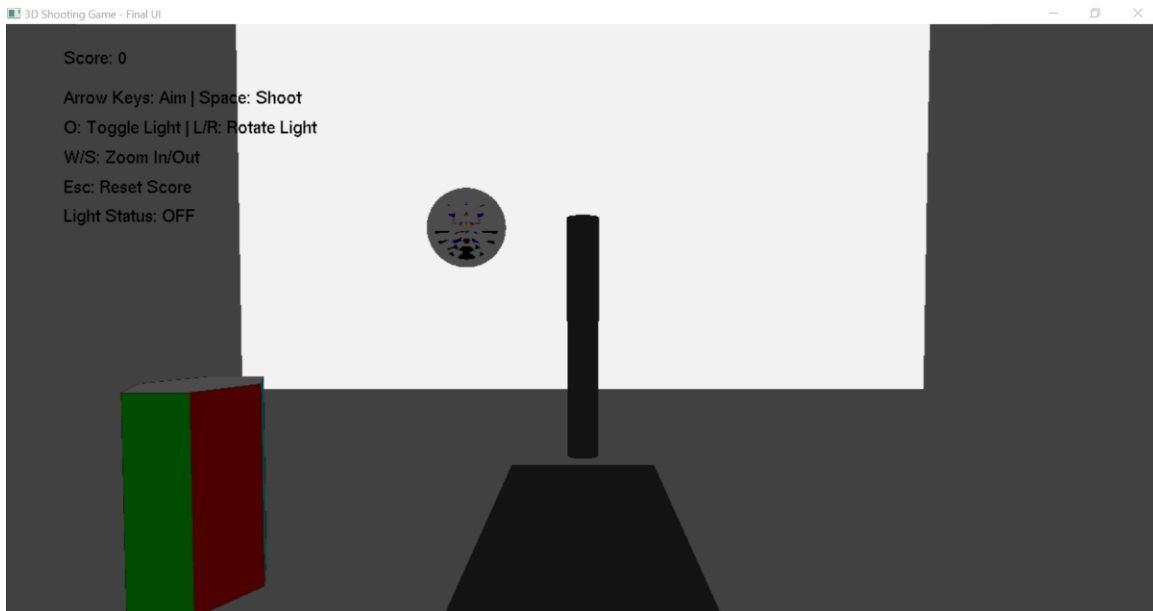
Student 1 (Abdullah Al Ovi):

- Draw: Implemented the room environment (drawRoom()), the complex hexagonal prism (drawHexagonalPrism()), and the volumetric light beam (drawLightBeam()).
- Animation: Handled the continuous left-right movement logic for the target.
- Controls: Implemented the shooting mechanism (Spacebar) and the score reset functionality (Esc).

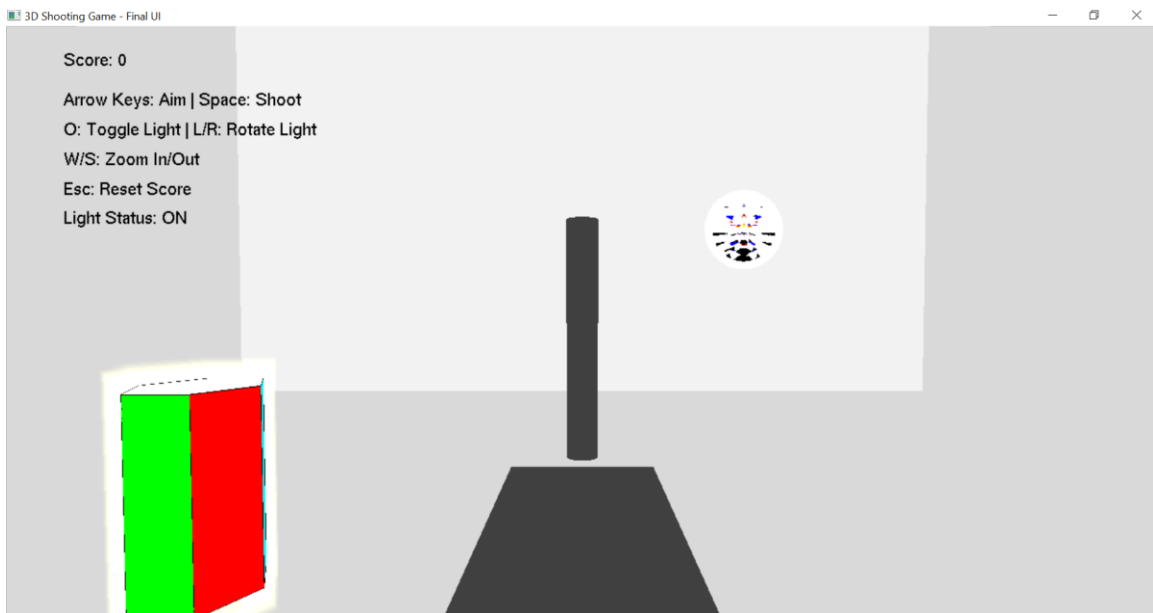
Student 2 (Jannatul Ferdous Mim):

- Draw: Designed the Olympic target (drawTarget()), the 3D gun model (drawGun()), and the aiming point system.
- Animation: Developed the bullet trajectory physics and the mathematical hit detection system (checkHit()).
- Controls: Implemented the light toggle (O), light rotation (L/R), and the aiming point movement (Arrow keys).

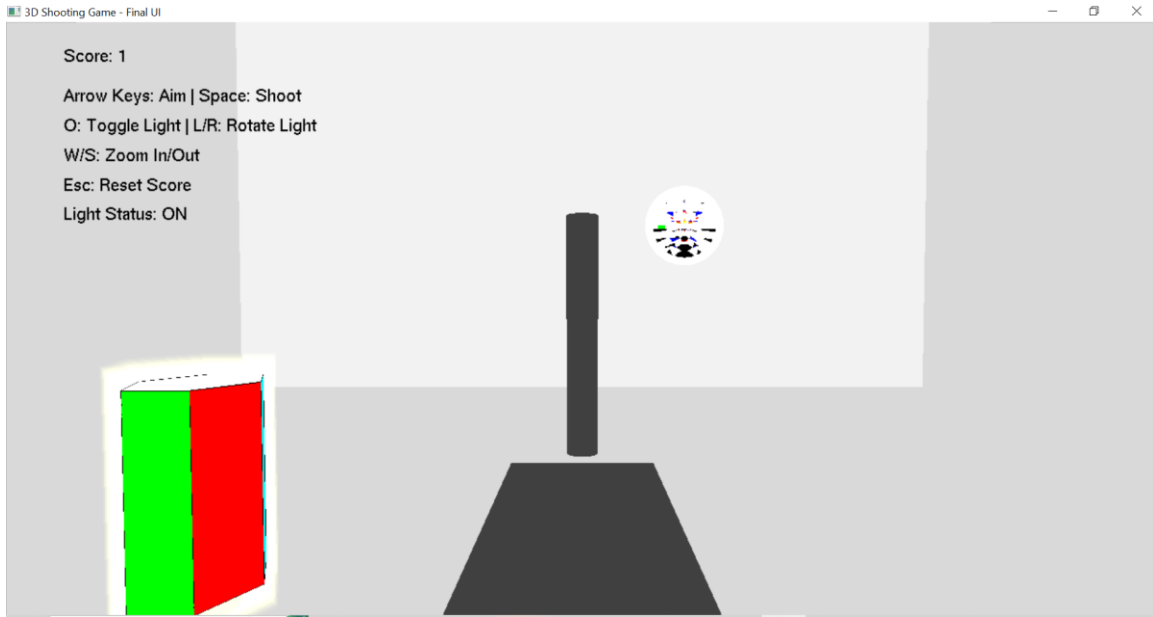
11. Results



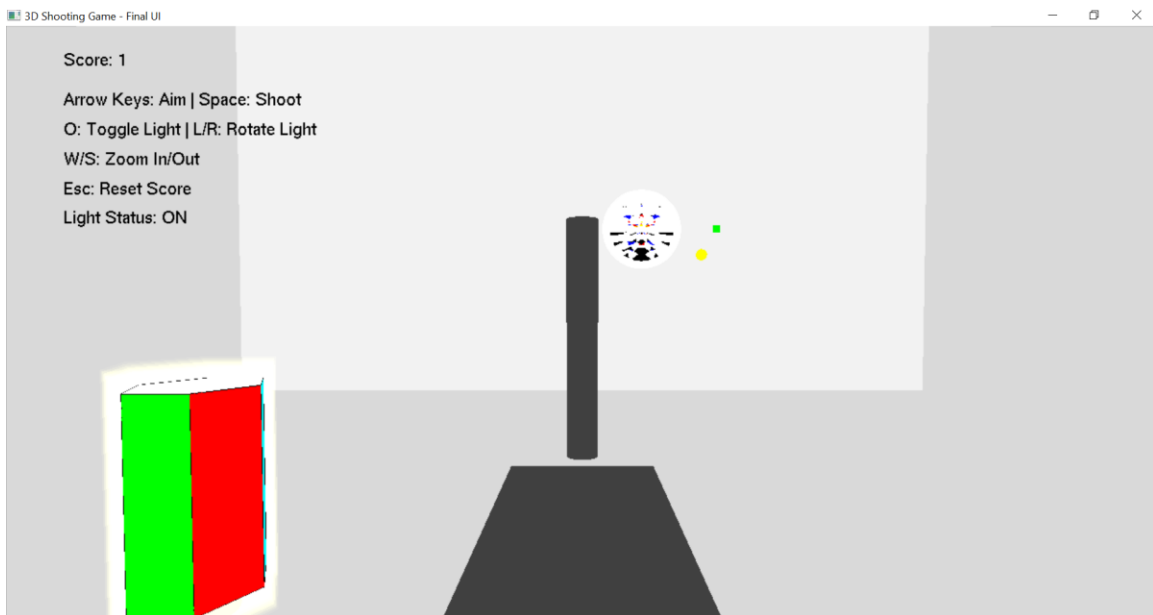
Screenshot 1: Game start (Light OFF, dimly lit room).



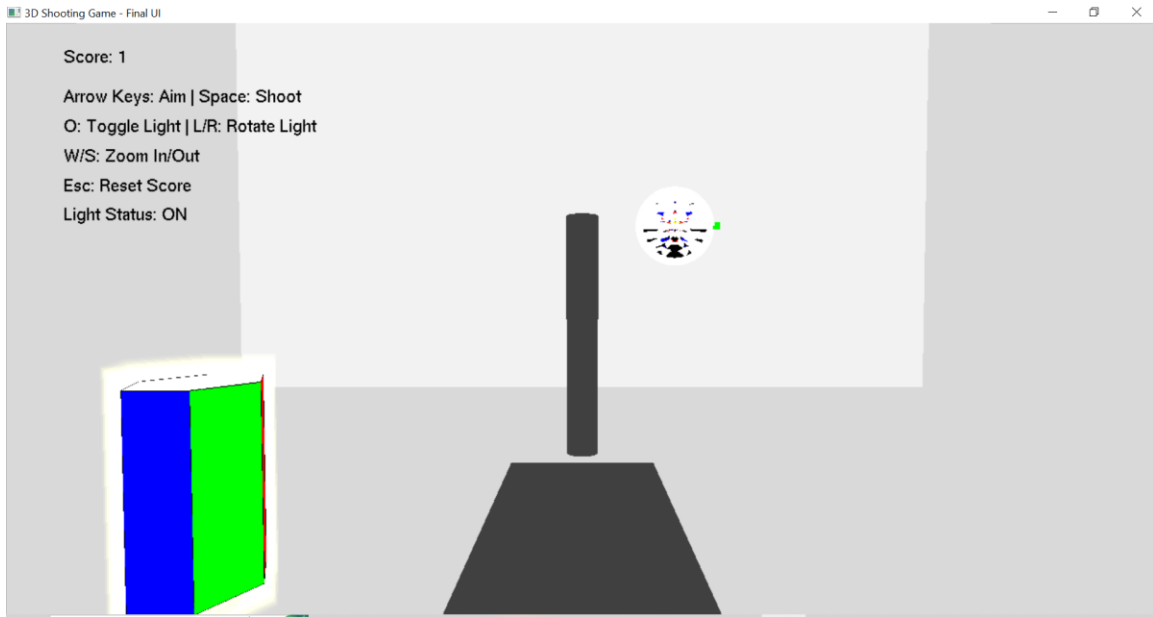
Screenshot 2: Light ON (Showing glowing hexagonal prism and volumetric beam).



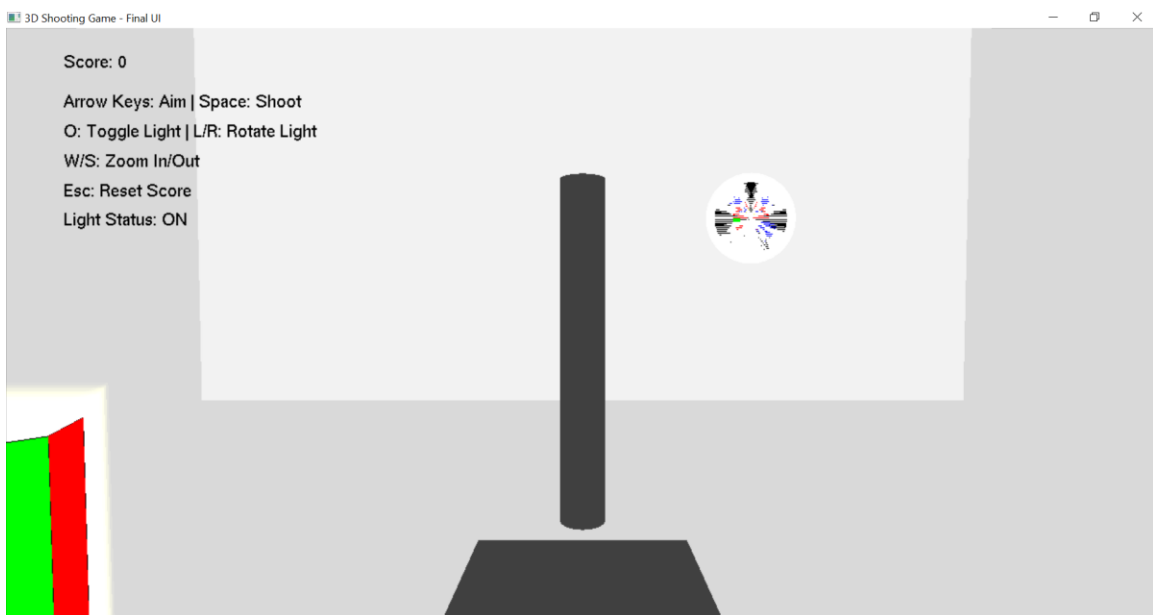
Screenshot 3: Shooting action (Bullet mid-air towards the target).



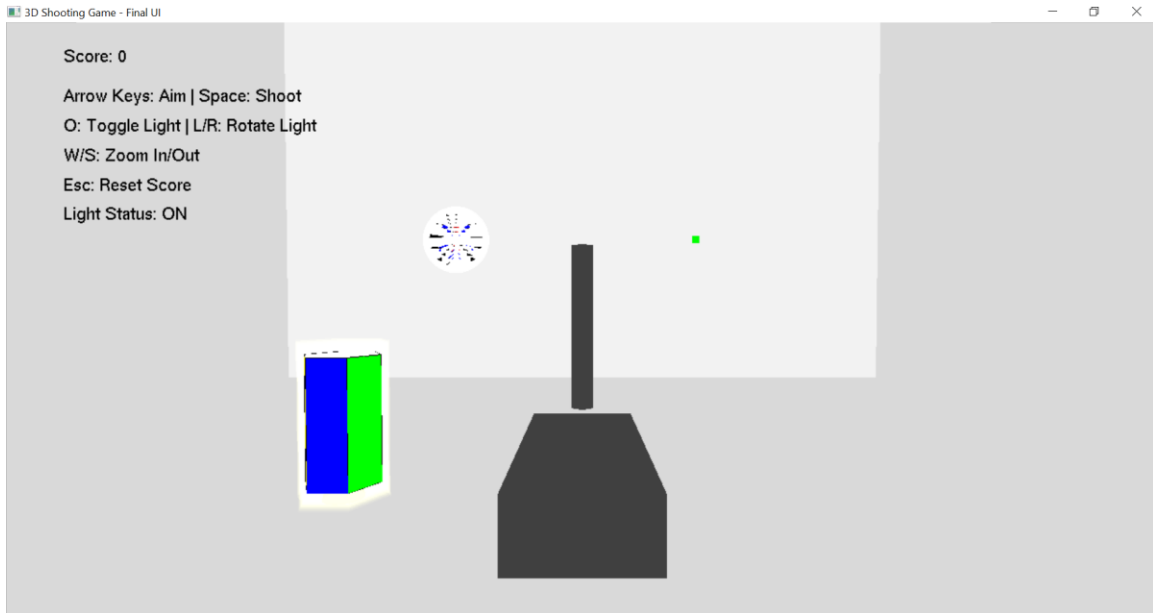
Screenshot 4: Miss hit



Screenshot 5: Rotation of 3D light



Screenshot 5: Zoom in



Screenshot 5: Zoom out

12. Conclusion

The 3D Shooting Game project successfully demonstrates the practical application of OpenGL and GLUT for interactive 3D graphics. By combining fundamental geometric transformations with advanced visual techniques like additive blending for glowing effects and procedural color mapping for ambient lighting, the project achieves a highly interactive and visually appealing result. The modular code structure and clear division of responsibilities ensured a smooth development process, fulfilling all course objectives.

Rubric

Course Outcome		Notes	Marks
CO1: Demonstrate the concept of computer graphics and ability to use the computer graphics technology.			25 marks
<u>Overall program structure:</u> - Free from error		5 marks	
<u>OpenGL Function:</u> - Apply functions of keyboard handling a. Rotation b. Translation c. Scaling <u>Texture Mapping:</u> - Apply functions of texture/color mapping for appropriate color		15 marks 5 marks	
CO2: Construct 2D and 3D graphics by implementing concepts of computer graphics and computer graphics programming.			25 marks
<u>Object:</u> -All required objects are provided		10 marks	
<u>Animation:</u> - Animation is implemented and suitable with the environment		10 marks	
<u>Camera / Lighting:</u> -Appropriate technique is implemented and suitable with the environment		5 marks	
CO3: Respond to instruction by listening actively and give feedback			25 Marks
-Camera moving -Animation -Answer the questions regarding basic of computer graphics		10 marks 10 marks 5 marks	
CO4: Work together effectively to achieve the same goal by building a good relationship and interaction among team members.			25 Marks
Student 1 Name: ID:	Objects (% of contribution) = 50	5 Marks	
	Camera (% of contribution) =60	5 Marks	
	Animation (% of contribution) =50	5 Marks	
	Report (% of contribution) =50	10 Marks	
Student 2 Name: ID:	Objects (% of contribution) = 50	5 Marks	
	Camera (% of contribution) =40	5 Marks	
	Animation (% of contribution) =50	5 Marks	
	Report (% of contribution)-50	10 Marks	

